Herald

Herald — Slack Channel Setup Guide

Field	Value
Revision	2
Status	**LIVE adapter — code complete; awaiting operator credentials for E127 live evidence** (HRD-115 open, operator-gated on a LIVE round-trip per §107.x; HRD-116 qaherald Slack client + HRD-118 e2e wiring already Fixed)
Spec ref	V4 §11 (Slack capabilities + Web API surface) + §32.2 (subscriber-reply continuous-transport loop) + §43 (multi-channel `pherald listen`)
HRD	HRD-115 (Slack channel adapter live integration), HRD-116 (qaherald Slack `MessengerClient`), HRD-118 (e2e E124–E131 multi-channel invariants)
Env vars	`HERALD_SLACK_BOT_TOKEN` (`xoxb-…`), `HERALD_SLACK_APP_TOKEN` (`xapp-…`, Socket Mode), `HERALD_SLACK_CHANNEL_ID` (`C0123ABCD`), `HERALD_SLACK_OPERATOR_USERNAME` (optional, attribution per §109)
Code	`commons_messaging/channels/slack/` (slack-go vendored as `github.com/slack-go/slack`) + `qaherald/internal/messenger/slack.go` (raw-HTTP QA client)

This guide walks you through every step needed to enable Slack in Herald — from creating the Slack app to verifying the live integration test passes. Once you complete these steps, set the env vars in your shell or `.env`, hand the tokens back to the operator, and we will close HRD-115 with live E127 evidence under `docs/qa/HRD-115-LIVE-<run-id>/`.

> **This is a risk-classified messenger service.** A leaked `xoxb-`/`xapp-` token can let an attacker read and post in your workspace. Read §8 — Token security before you paste any token anywhere. Herald never commits tokens and the adapter redacts them out of every error string (see §8).

Table of contents

Pre-requisites

- A Slack workspace where **you are allowed to install apps**. Some workspaces restrict app installation to admins or require admin approval; if so, you'll submit a request and an admin approves it (this is normal — see §9).
- The Herald checkout at `/Users/milosvasic/Projects/Herald` (or wherever you cloned it).
- Either:
  - `podman` or `docker` available locally (for the quickstart compose stack), OR
  - a reachable Postgres instance per the §"Postgres" entries in `../OPERATOR_CREDENTIALS.md`.
- Go 1.25+ (the live test runs `go test`).

1 — How Slack fits into Herald (architecture)

Herald's Slack support is the **second** concrete `channels.Channel` implementation (Telegram was first, in Wave 6). It lives at `commons_messaging/channels/slack/` and registers itself with the channels registry under the name `"slack"` via an `init()` so `pherald listen --channels slack` resolves it at runtime.

The adapter uses **two different transports** — this is the single most important thing to understand before you provision tokens:

Direction	Transport	Slack methods	Token required
Outbound (Herald → Slack)	Slack **Web API** (HTTPS request/response)	`chat.postMessage`, `files.uploadV2`, `files.info`, `auth.test`	bot token `xoxb-…`
Inbound (Slack → Herald)	**Socket Mode** (a persistent outbound WebSocket — the Slack equivalent of Telegram's `getUpdates` long-poll)	Events API over the WebSocket (`message` events)	**both** the bot token `xoxb-…` AND an app-level token `xapp-…`

Concretely:

- `Adapter.Send` / `Adapter.SendReply` (in `send.go`) call `chat.postMessage`. The `Receipt.ChannelMsgID` carries the real Slack `ts` returned by the server — not a synthetic Herald UUID. Threaded replies set `thread_ts` (the `MsgOptionTS` slack-go option) so a reply threads under the parent message (§32.9 reply anchor). Attachments fan out via `files.uploadV2` with the same `thread_ts`.
- `Adapter.BotSelfIdentity` / `Adapter.HealthCheck` (in `selfidentity.go` / `healthcheck.go`) call `auth.test`. The returned `user_id` is the bot's own id — Herald caches it and uses it as the §32.9 anti-echo-loop anchor so the

bot never reacts to its own posts. A degenerate `ok=true` response with an empty `user_id` is rejected as an echo-loop hazard.
- `Adapter.Subscribe` (in `subscribe.go`) runs the **Socket Mode** event loop. It **requires** the app-level token (`xapp-…`); constructing it without one is a deterministic boot error (`slack.Subscribe: app-level token (xapp-…) required for Socket Mode`). Each inbound `message` event becomes a `commons.InboundEvent`; the bot's own echoes are dropped via the cached self-identity; file shares trigger content-addressed downloads (`files.info` → `url_private_download`) into the per-channel inbox.

There is a **separate** Slack client used only by the QA bot: `qaherald/internal/messenger/slack.go`. It is a small raw-HTTP client (NOT the slack-go SDK) that drives `pherald` ? Slack round-trip automation for §107.x evidence. It uses `conversations.history` with an `oldest=<ts>` watermark in place of a `getUpdates` equivalent (Slack has none). The live test you will run in §7 exercises this QA client.

> **Negative finding (honest divergence).** The qaherald QA client currently ships the **deprecated single-step** `files.upload` (Slack deprecated it in 2025 in favor of the two-step V2 flow `files.getUploadURLExternal` + `files.completeUploadExternal`). This is documented in the `slack.go` header as known-future-cleanup; it is functionally equivalent against the hermetic test seam and swappable without API breakage. The **production channel adapter** (`commons_messaging/channels/slack/`) already uses the current `files.uploadV2`. If your live attachment upload via qaherald fails with a deprecation error, this is the cause — text Send/Reply (the E127 path) is unaffected.

2 — Step 2 — Create the Slack app

Slack apps are managed at **https://api.slack.com/apps**.

1. Open **https://api.slack.com/apps** in a browser and sign in to the workspace where you'll install Herald.
2. Click **Create New App**.
3. Choose **From scratch** (the manifest route also works, but "from scratch" matches this guide step-by-step).
4. Enter an **App Name** — e.g. `Herald Notifier`, `Herald CI`, `MyProject Herald`.
5. Pick the **workspace** to develop the app in (the workspace whose channel you'll post to).
6. Click **Create App**. You land on the app's **Basic Information** page.

> The app is owned by your workspace. Anyone with the app's tokens can act as the bot — keep the management page and tokens private.

3 — Step 3 — OAuth scopes + install → get the `xoxb-` bot token

The bot token (`xoxb-…`) is what Herald uses for **all outbound** calls. You grant it specific **bot token scopes**, then install the app to mint the token.

1. In the left sidebar, open **OAuth & Permissions**.
2. Scroll to **Scopes → Bot Token Scopes** and click **Add an OAuth Scope** for each scope below.

The scopes Herald's adapter actually needs (verified 2026-05-31 against the official scope reference — see Sources verified):

Scope	What it grants (official wording)	Why Herald needs it
`chat:write`	"Send messages as your Slack app"	**Required.** Every outbound message (`chat.postMessage`) — `Send`, `SendReply`, and the qaherald QA client. `auth.test` works with no scope, but a usable bot needs this.
`channels:history`	"View messages and other content in **public** channels that your Slack app has been added to"	Inbound (Socket Mode `message` events) + the qaherald QA client's `conversations.history` polling **when the target is a public channel**.
`groups:history`	"View messages and other content in **private** channels that your Slack app has been added to"	Same as above **when the target is a private channel**. Add this only if your `HERALD_SLACK_CHANNEL_ID` points at a private channel.
`files:read`	View files shared in channels the app is in	Inbound attachment download (`files.info` → `url_private_download`) when subscribers share files.
`files:write`	Upload/edit/delete files as the app	Outbound attachment fan-out (`files.uploadV2` in the adapter; `files.upload` in qaherald). Needed only if you send attachments.

Optional / situational:

- `chat:write.public` — "Send messages to channels your Slack app **isn't a member of**" (requires `chat:write` too). Add this **only** if you want Herald to post to a public channel **without** inviting the bot (§5). The simpler, recommended path is to invite the bot and skip this scope.
- `im:history` / `mpim:history` — read message history in **DMs** / **multi-person DMs**. Add these only if your target "channel" is a DM or group DM (the `oldest`-watermark `conversations.history` poll needs the matching `*:history` scope per conversation type).
- `channels:read` / `groups:read` — list/inspect channel metadata. The adapter does not require these for the core Send/Reply/Subscribe path; the qaherald `Preflight` self-check calls `conversations.info` and `conversations.members`, which need `channels:read` / `groups:read` if you run the preflight. They are NOT needed for E127.

  Minimum for E127 (text Send + reply round-trip in a channel you invite the bot to): `chat:write` + `channels:history` (public channel) **or** `groups:history` (private channel). Everything else is for attachments, DMs, or preflight.

1. Still on **OAuth & Permissions**, scroll to the top and click **Install to Workspace** (or **Request to Install** if your workspace requires admin approval — see §9).
2. Review the permission screen and click **Allow**.
3. You're returned to **OAuth & Permissions**. Under **OAuth Tokens for Your Workspace**, copy the **Bot User OAuth Token** — it starts with `xoxb-`:

```
xoxb-<13-digit-team>-<13-digit-bot>-<24-char-secret>
```

4. **Copy it immediately** to a password manager / secrets vault. This is `HERALD_SLACK_BOT_TOKEN`.

   Whenever you change scopes later, you **must reinstall** the app (repeat steps 3–5) for the new scopes to take effect — otherwise calls fail with `missing_scope` (§9).

4 — Step 4 — Enable Socket Mode → get the `xapp-` app-level token

Socket Mode is Herald's **inbound** transport — the persistent WebSocket that delivers subscriber messages. It needs a **separate** app-level token (`xapp-…`), distinct from the bot token. (If you only need **outbound** notifications and never inbound replies, you can skip this step and leave `HERALD_SLACK_APP_TOKEN` unset; `Send`/`HealthCheck` work without it. `Subscribe` will refuse to boot without it.)

1. In the left sidebar, open **Socket Mode**.
2. Toggle **Enable Socket Mode** on. (Official wording: "Toggle the Enable Socket Mode button to turn on receiving payloads via WebSockets.")
3. Slack prompts you to generate an **app-level token**. (You can also do this under **Basic Information → App-Level Tokens → Generate Token and Scopes**.)
4. Give the token a name (e.g. `socket`) and add the scope `connections:write` — the official scope that "Grants permission to generate websocket URIs and connect to Socket Mode" (verified 2026-05-31). This is the **only** scope an app-level token needs for Herald.
5. Click **Generate**. Copy the token — it starts with `xapp-`:

   ```
   xapp-1-<app-id>-<config-token-id>-<64-char-secret>
   ```

6. **Copy it immediately** to your vault. This is `HERALD_SLACK_APP_TOKEN`.
7. Open **Event Subscriptions** in the sidebar and ensure it is **On**. Under **Subscribe to bot events**, add `message.channels` (public channels) and/or `message.groups` (private channels) so the bot receives the `message` events that drive `Subscribe`. (Over Socket Mode you do **not** configure a Request URL — events arrive on the WebSocket.)

   Negative finding (no signing secret needed). A common Slack setup uses the **Events API over HTTPS webhooks**, which requires a **Signing Secret** to verify request signatures. Herald uses **Socket Mode**, not HTTPS webhooks, so **no signing secret is read by the code**. (An earlier draft of this guide reserved `HERALD_SLACK_SIGNING_SECRET` — that env var is **not** read by the adapter or any test. Do not set it expecting an effect.) Likewise the old `HERALD_SLACK_DEFAULT_CHANNEL` name is **not** used — the real default-channel var is `HERALD_SLACK_CHANNEL_ID` (§6).

5 — Step 5 — Find the channel ID + invite the bot

Slack channels are addressed by **ID** (e.g. `C0123ABCD`), not by name — names are renamable, IDs are stable. Herald sends to the ID in `HERALD_SLACK_CHANNEL_ID`.

Find the channel ID:

1. In the Slack desktop or web client, open the target channel.

2. Click the channel name at the top to open its details, scroll to the bottom of the **About** tab — the **Channel ID** (e.g. `C0123ABCD` for a public channel, `G…` for some private channels) is shown with a copy button.
    - Alternatively, copy the channel's link (right-click the channel → **Copy link**); the ID is the last path segment, e.g. `…/archives/C0123ABCD`.
3. Paste it somewhere safe. This is `HERALD_SLACK_CHANNEL_ID`.

Invite the bot to the channel (required unless you granted `chat:write.public` for a public channel):

1. In the channel's message box type:

   ```
   /invite @Herald Notifier
   ```

   (use your app's bot display name) and send it. Slack adds the bot to the channel.

   If you skip the invite and the bot is not in the channel, `chat.postMessage` returns `not_in_channel` (§9). Inviting the bot is the simplest fix; `chat:write.public` is the alternative for public channels only.

6 — Step 6 — Provide the credentials to Herald (env vars)

Herald reads exactly these env vars for Slack (verified against `pherald/cmd/pherald/listen.go` and `scripts/e2e_bluff_hunt.sh`):

Env var	Value	Used by
`HERALD_SLACK_BOT_TOKEN`	the `xoxb-…` bot token (§3)	outbound (`chat.postMessage`, `auth.test`, `files.*`); E127 live test
`HERALD_SLACK_CHANNEL_ID`	the target channel ID, e.g. `C0123ABCD` (§5)	default outbound destination; E127 live test
`HERALD_SLACK_APP_TOKEN`	the `xapp-…` app-level token (§4)	inbound **Socket Mode** (`Subscribe`)
`HERALD_SLACK_QA_USER_TOKEN`	a Slack **user** OAuth token (`xoxp-…`) for a member of the channel (§7a)	the §11.4.98 self-driving inbound **round-trip** QA harness (`TestSlack_LiveRoundTrip`) — drives the *user* side
`HERALD_SLACK_OPERATOR_USERNAME`	the operator's Slack `@username`	participant attribution / tagging (§109, the `HERALD_<CHANNEL>_OPERATOR_USERNAME` family)

Where to put them. Per CLAUDE.md credential rules and `../OPERATOR_CREDENTIALS.md`:

- Either **export them in your shell** (`~/.bashrc` / `~/.zshrc`) — these load **first** and take precedence — or put them in `.env` (git-ignored) as a **fallback** that never overrides an exported shell var.
- A committed `.env.example` documents the names; never commit real values.

Example `.env` block (values are placeholders):

```
HERALD_SLACK_BOT_TOKEN=xoxb-<13-digit-team>-<13-digit-bot>-<24-
        char-secret>
HERALD_SLACK_CHANNEL_ID=C0123ABCD
# Only needed if you want inbound (Socket Mode) replies:
HERALD_SLACK_APP_TOKEN=xapp-1-<app-id>-<config-token-id>-<secret>
# Only needed for the §11.4.98 self-driving inbound round-trip QA
        test (§7a):
# HERALD_SLACK_QA_USER_TOKEN=xoxp-<user-oauth-token>
# Optional — attribution/tagging:
# HERALD_SLACK_OPERATOR_USERNAME=@yourhandle
```

> **Resolution order (load-bearing).** Exported shell vars win; `.env` is fallback only. If a stale shell export shadows your `.env`, the `.env` value is silently ignored — `unset HERALD_SLACK_BOT_TOKEN` (or open a fresh shell) if you change credentials.

7 — Step 7 — Verify the live round-trip (E127 / `TestSlack_Live_Send`)

E127 is the canonical **live Slack Send + reply round-trip** invariant in `scripts/e2e_bluff_hunt.sh`. It is **triple-guarded** to prevent a silent "no tests to run" PASS-bluff — it only runs when **all three** hold:

1. `HERALD_SLACK_BOT_TOKEN` is set,
2. `HERALD_SLACK_CHANNEL_ID` is set, **and**
3. a `func TestSlack_Live_Send` exists in `qaherald/internal/messenger/*_test.go`.

If any is missing, E127 prints an explicit **SKIP-with-reason** (§11.4.3) citing the hermetic transcripts as the standing evidence — it never silently passes.

The exact command the operator runs (mirrors the e2e check verbatim):

```
cd /Users/milosvasic/Projects/Herald
HERALD_SLACK_BOT_TOKEN='xoxb-…' \
HERALD_SLACK_CHANNEL_ID='C0123ABCD' \
go test -tags=integration -count=1 -run TestSlack_Live_Send ./
        qaherald/internal/messenger/... -timeout=120s -v
```

What it proves (the real assertions live in `qaherald/internal/messenger/slack.go` + its test):

- `Send` hits `chat.postMessage` and returns the **real Slack** `ts` (a dotted-float string like `1654.0001`), not a synthetic id — a no-op Send would return an empty `ts` and FAIL the §107 `ErrEmptyResponse` guard.
- The reply path threads under the parent `ts`.
- You should see the test message **arrive in your Slack channel**.

Expected:

```
--- PASS: TestSlack_Live_Send (Xs)
PASS
ok      github.com/vasic-digital/herald/qaherald/internal/messenger   X.XXs
```

Or, run the whole smoke and watch the E127 line:

```
bash scripts/e2e_bluff_hunt.sh 2>&1 | grep -E 'E12[4-9]|E13[01]'
```

Where the evidence lands (§107.x). Capture the full bidirectional transcript (your message + the bot's reply, request payloads + full response bodies) and commit it under:

```
docs/qa/HRD-115-LIVE-<run-id>/
```

(e.g. `docs/qa/HRD-115-LIVE-20260531T120000Z/transcript.txt`). Hand this back to the operator — this is what closes HRD-115 from "code complete, awaiting live evidence" to **Fixed**. Until it lands, HRD-115 stays open and E127 stays SKIP (the hermetic transcripts at `docs/qa/HRD-115-20260528T080000Z/` + `docs/qa/HRD-116-20260528T090000Z/` are the standing §107 evidence).

> **Note on the test's current state.** As shipped, `TestSlack_Live_Send` is a **future operator-evidence task** — the e2e guard checks for its *presence* precisely so that running `-run TestSlack_Live_Send` against a missing test cannot silently report "no tests to run" as a PASS. If the test is not yet present in your checkout, E127 will SKIP with that exact reason; the hermetic `TestSlackClientSendCrossesWire` / `TestBuilderSlack` tests (E126/E131) already exercise the wire path against an httptest server.

7a — Step 7a — The fully-automated inbound round-trip (§11.4.98 / `TestSlack_LiveRoundTrip`)

§7 above (E127 / `TestSlack_Live_Send`) proves the **send** side — a real `chat.postMessage` returns a real `ts`. The **inbound round-trip** — a subscriber message arriving over Socket Mode, being dispatched to Claude Code, and getting a reply back — is proven by a separate, fully-self-driving harness: `TestSlack_LiveRoundTrip` (in `qaherald/internal/lifecycle/`, build tag `integration`). This is the §11.4.98-compliant analog of the Telegram/MTProto closed-loop harness; it is what closes **HRD-115** from "send-side proven" to a fully-proven round-trip.

Why a separate QA *user* token is required (the echo-wall). A Slack **bot never receives its own messages** over Socket Mode — the platform deliberately suppresses self-echoes (and Herald's `§32.9` self-filter drops them defensively too). So the pherald bot can never see a message authored by *itself*. To drive a real inbound message the harness needs a **second, distinct identity**:

a Slack **user** (`xoxp-…`) that posts into the channel as a human would. The bot then receives *that* user's message, exactly as it would in production.

`HERALD_SLACK_QA_USER_TOKEN` is that user OAuth token:

- **Type:** a **user** token (`xoxp-…`), NOT a bot token. (Install the app with a **User Token Scope**, or use a token minted for a real workspace user.)
- **Scopes:** `chat:write` (to post the probe as the user) + `channels:history` (to read the bot's reply back via `conversations.history`). Use `groups:history` instead of `channels:history` if `HERALD_SLACK_CHANNEL_ID` is a **private** channel.
- **Membership:** the user behind the token **must be a member of** `HERALD_SLACK_CHANNEL_ID` (post + history both require it). Invite the user (or yourself) to the channel first.

What the harness does, with ZERO human action during the run:

1. Builds `pherald` and spawns `pherald listen --channels slack` (wired to `HERALD_SLACK_BOT_TOKEN` + `HERALD_SLACK_APP_TOKEN`, journaling to a temp `--qa-out-dir`), with a **dedicated Claude session name** so it never collides with your dev session (§11.4.98 rule 2).
2. Waits for the Socket Mode connection, then **posts a unique nonce-bearing probe as the QA user** (`chat.postMessage` via `HERALD_SLACK_QA_USER_TOKEN`).
3. Asserts the autonomy chain via the pherald JSONL journal — inbound message (`channel:slack`) carrying the probe → `cc.dispatch` → `cc.reply` — and, as a bonus, reads the bot's reply back via `conversations.history`.
4. SIGTERMs the listen subprocess and asserts a clean exit.
5. Writes a **redacted** transcript (tokens + `wss://` scrubbed) under `docs/qa/HRD-115-LIVE-roundtrip-<TS>/`.

The exact command:

```
cd /Users/milosvasic/Projects/Herald
HERALD_SLACK_BOT_TOKEN='xoxb-…' \
HERALD_SLACK_APP_TOKEN='xapp-…' \
HERALD_SLACK_CHANNEL_ID='C0123ABCD' \
HERALD_SLACK_QA_USER_TOKEN='xoxp-…' \
go test -tags=integration -count=1 -run
        TestSlack_LiveRoundTrip ./qaherald/internal/
        lifecycle/... -timeout=300s -v
```

Without all four credentials (and a `claude` binary on `PATH` or `HERALD_CLAUDE_BIN`), the test **SKIPs with an explicit reason** (§11.4.3) — never a fake pass, never a manual-action prompt.

7b — Step 7b — Threading & thread context

Herald keeps Slack conversations **in-thread** and makes its replies **aware of the thread they belong to**. Two behaviours, both live (operator mandate 2026-06-02):

Replies land in-thread (`thread_ts`)

Every reply Herald sends in response to a subscriber message is delivered **into a Slack thread**, never as a loose top-level post:

- When the subscriber's message is **already inside a thread**, Herald reads that thread's root (the message's `thread_ts`) and posts its reply into the **same thread** — the conversation stays where the subscriber started it.
- When the subscriber's message is a **top-level channel message** (no `thread_ts`), Herald replies **on that message**, starting a thread anchored to it.

Under the hood the channel-agnostic dispatcher (`pherald/internal/inbound/dispatcher.go`, `extractReplyToID`) **prefers the inbound message's** `thread_ts` and otherwise falls back to the message's own `ts`; the Slack adapter maps that string straight onto the `thread_ts` field of `chat.postMessage`. The result: a subscriber who replies inside an existing thread is answered **in that same thread**.

Inbound threaded messages are processed

A subscriber reply made **within a thread** (or a reply that creates a thread) is a normal inbound message — Herald receives it over Socket Mode, dispatches it to Claude Code, and routes the answer back into the same thread. There is no "threads are ignored" gap: threaded and top-level messages are handled identically except that the reply destination follows the thread.

Thread context binds replies to the thread's meaning

When an inbound message belongs to a thread, the Slack adapter gathers the thread's **prior messages** so Claude can answer in context rather than in isolation:

- The adapter calls Slack's `conversations.replies` for the thread root (`commons_messaging/channels/slack/thread_context.go`, `fetchThreadContext`), takes the prior messages in oldest→newest order (bounded to the most recent 20 so a long thread cannot bloat the envelope), and **excludes the current message** (it is already the message being answered).
- Those prior messages are attached to the inbound event as `commons.ThreadContext` and rendered into the Claude Code dispatch envelope (`renderThreadContext`), so the reply is **bound by the thread's meaning** — a contribution to the discussion, made only when the thread's context warrants it.
- Context-gathering is **non-fatal**: if `conversations.replies` errors or returns nothing, the adapter logs and dispatches the message **without** prior context rather than dropping it. A context-fetch failure never silences a subscriber.

Scopes required for reading the thread

Reading thread history with `conversations.replies` needs the same history scope as ordinary inbound:

- `channels:history` — when `HERALD_SLACK_CHANNEL_ID` is a **public** channel.
- `groups:history` — when it is a **private** channel.

These are the scopes you already added in §3 for inbound; no additional scope is needed for threading. (The §7a `TestSlack_LiveRoundTrip` QA harness uses `channels:history` / `groups:history` on the QA **user** token to read the bot's reply back.)

Live evidence

The in-thread round-trip is proven by `TestSlack_LiveRoundTrip` (§7a), whose three legs include an **in-thread subscriber reply answered in-thread**. The captured, token-redacted transcript lands under:

`docs/qa/HRD-115-LIVE-roundtrip-<TS>/`

(e.g. `docs/qa/HRD-115-LIVE-roundtrip-2026-06-02T09-24-28Z/`). This is the standing §107.x evidence for Slack threading + thread-context awareness.

8 — Token security (§107 redaction, rotation)

- **Never commit a token.** Not in code, a PR, a commit message, a committed `.env`, a `docs/qa/` transcript, or any indexed location. `.env` is git-ignored; keep it that way.
- **Tokens never leak to logs.** The qaherald Slack client deliberately keeps the token out of every error string — errors name the API method (`chat.postMessage failed: …`) but never the URL or token — and a belt-and-braces `sanitizeError(msg, token)` scrubs any accidental occurrence to `[REDACTED]`. The test `TestSlack_Send_ErrorDoesNotLeakToken` pins this with a planted sentinel `xoxb-<planted-sentinel-must-not-leak-7777>` and asserts it never appears in error text. The production channel adapter likewise carries the bot token only in the `Authorization: Bearer` header, never in URLs.
- **Two tokens, two blast radii.** `xoxb-` (bot) can read/post per its granted scopes; `xapp-` (app-level) can open Socket Mode WebSockets. Rotate **both** if either leaks.
- **Rotate by regenerating.** Bot token: **OAuth & Permissions → Reinstall / rotate** (or revoke + reinstall). App-level token: **Basic Information → App-Level Tokens → revoke** and generate a new one. Git-history scrubbing alone does **not** invalidate a leaked Slack token — you must revoke/rotate on Slack's side.
- **Least privilege.** Grant only the scopes §3 lists for your actual use. Skip `files:write`/`files:read` if you don't send/receive attachments; skip `chat:write.public` if you invite the bot.

9 — Step 9 — Troubleshooting

All Slack Web API failures return a JSON envelope `{"ok": false, "error": "<reason>"}`; Herald surfaces `<reason>` in the wrapped Go error (token-redacted). The error strings below are the official Slack reason codes (verified 2026-05-31).

Symptom / error	Cause	Fix
`invalid_auth`	"Some aspect of authentication cannot be validated." The `xoxb-` token is wrong, revoked, or for the wrong workspace.	Re-copy the **Bot User OAuth Token** from **OAuth & Permissions**. Check for trailing whitespace. Confirm the workspace matches.
`not_authed`	"No authentication token provided." `HERALD_SLACK_BOT_TOKEN` is empty / unset.	Confirm the env var is set in the **same shell** that runs the test (`echo ${HERALD_SLACK_BOT_TOKEN:+set}`). Remember exported shell vars override `.env`.
`not_in_channel`		

Symptom / error	Cause	Fix
	"Cannot post … to a channel they are not in." The bot was never invited to `HERALD_SLACK_CHANNEL_ID`.	`/invite @<bot name>` in the channel (§5), **or** add `chat:write.public` for a public channel and reinstall.
`channel_not_found`	"Value passed for `channel` was invalid." Wrong / mistyped channel ID, or a private channel the bot can't see.	Re-copy the **Channel ID** from the channel's About panel (§5). For a private channel the bot must be invited AND you need `groups:history`.
`is_archived`	"Channel has been archived."	Unarchive the channel or point `HERALD_SLACK_CHANNEL_ID` at a live channel.
`missing_scope`	"The token … is not granted the specific scope permissions required." You added a scope but didn't reinstall, or never added it.	Add the scope (§3), then **Reinstall to Workspace** so the new scope takes effect. `chat.postMessage` needs `chat:write`; `conversations.history` needs the matching `*:history` scope per conversation type.
Socket Mode won't connect / `Subscribe` **errors immediately with "app-level token (xapp-…) required"**	`HERALD_SLACK_APP_TOKEN` is unset, or the app-level token lacks `connections:write`, or Socket Mode is toggled off.	Generate an `xapp-` token with `connections:write` (§4), set `HERALD_SLACK_APP_TOKEN`, and ensure **Socket Mode** is enabled.
Inbound events never arrive (Socket Mode connects but no messages)	**Event Subscriptions** off, or the bot isn't subscribed to `message.channels` / `message.groups`, or the bot isn't in the channel, or it lacks `channels:history` / `groups:history`.	Enable Event Subscriptions, add `message.channels`/ `message.groups`, invite the bot, add the matching `*:history` scope and reinstall.
"empty user_id (echo-loop hazard)" from HealthCheck/ BotSelfIdentity	`auth.test` returned `ok=true` but no `user_id` — a degenerate/ deactivated token.	Reinstall the app to mint a fresh bot token; confirm the bot user is not deactivated.
App won't install ("requires admin approval")	Workspace restricts app installation.	Click **Request to Install**; a workspace admin approves it. This is a workspace policy, not a Herald issue.
`files.upload` **fails / deprecation error (qaherald only)**	qaherald ships the deprecated single-step `files.upload` (§1 negative finding).	Text Send/Reply (E127) is unaffected. For attachments use the channel adapter's `files.uploadV2`; the qaherald V2 shim is known-future-cleanup.
Live test SKIPs despite tokens set	One of the E127 triple-guards is unmet (missing `HERALD_SLACK_CHANNEL_ID`, or `TestSlack_Live_Send` not present in the checkout).	Set all three; confirm `grep -l 'func TestSlack_Live_Send' qaherald/internal/ messenger/*_test.go`. The

Symptom / error	Cause	Fix
		SKIP reason text states exactly which guard failed.

10 — Step 10 — Pre-deploy operator audit checklist

Run through this before handing credentials to the operator or enabling Slack in production:

- [ ] App created at https://api.slack.com/apps in the correct workspace.
- [ ] Bot Token Scopes include at least `chat:write` (+ `channels:history` or `groups:history` for the target channel type); attachment scopes (`files:read`/`files:write`) only if used.
- [ ] App **installed** (or reinstalled after the last scope change) — `xoxb-` token copied.
- [ ] (Inbound only) Socket Mode **enabled**; app-level `xapp-` token generated with `connections:write`; Event Subscriptions on with `message.channels`/`message.groups`.
- [ ] Channel ID copied; **bot invited** to the channel (or `chat:write.public` granted for a public channel).
- [ ] `HERALD_SLACK_BOT_TOKEN`, `HERALD_SLACK_CHANNEL_ID` (and `HERALD_SLACK_APP_TOKEN` if inbound) set in shell **or** `.env`; no real values committed; `.env` is git-ignored.
- [ ] No `HERALD_SLACK_SIGNING_SECRET` / `HERALD_SLACK_DEFAULT_CHANNEL` set expecting an effect (those names are **not** read by the code).
- [ ] E127 / `TestSlack_Live_Send` runs (or SKIPs with a precise reason); the message arrives in the channel.
- [ ] Bidirectional transcript captured under `docs/qa/HRD-115-LIVE-<run-id>/` (§107.x) and handed to the operator.
- [ ] Tokens stored in a vault; rotation procedure (§8) understood.

11 — Step 11 — Spec + code references

- **Spec**: V4 §11 (Slack channel capabilities + Web API surface), §32.2 (subscriber-reply continuous-transport loop — Socket Mode is Slack's parity with Telegram long-poll), §43 (multi-channel `pherald listen`).
- **HRDs**: HRD-115 (Slack channel adapter live integration — **open**, operator-gated on LIVE `docs/qa/HRD-115-LIVE-*/` evidence per §107.x); HRD-116 (qaherald Slack `MessengerClient` — **Fixed**); HRD-118 (e2e E124–E131 multi-channel invariants — **Fixed**).

- **Code (production channel adapter)** — `commons_messaging/channels/slack/`:
  - `slack.go` — `Adapter` struct + `slack://` URL parser + `Capabilities` + `init()` registry wiring
  - `send.go` — `Send` / `SendReply` / `SendReplyGeneric` (`chat.postMessage` + `thread_ts` + `files.uploadV2` fan-out)
  - `subscribe.go` — `Subscribe` Socket Mode event loop (`xapp-` required) + `dispatchMessageEvent` + self-echo drop
  - `selfidentity.go` — `BotSelfIdentity` (`auth.test`, cached user_id, §32.9 anchor)
  - `healthcheck.go` — `HealthCheck` (`auth.test` token-validity probe)
  - `attachments.go` — content-addressed `DownloadAttachment` (`files.info` → `url_private_download`)
- **Code (qaherald QA client)** — `qaherald/internal/messenger/slack.go` — raw-HTTP `SlackClient` implementing `MessengerClient` (Send / SendPhoto / SendDocument / SendVoice / GetUpdates via `conversations.history` watermark / WaitForReply / Download / Preflight / Me / Close); `sanitizeError` token redaction.
- **Tests**:
  - `commons_messaging/channels/slack/*_test.go` — hermetic httptest suite (E126: `TestSlackSatisfiesChannel`, `TestSlackRegistryWiring`, `TestSlackBotSelfIdentityViaAuthTest`, `TestSlackSendCrossesWireWithText`)
  - `qaherald/internal/messenger/slack_test.go` — `TestSlack_Send_ErrorDoesNotLeakToken` (§8 redaction), builder tests (E131)
  - `scripts/e2e_bluff_hunt.sh` — E124–E131 (E127 = LIVE Slack, triple-guarded)
- **Vendored SDK**: `github.com/slack-go/slack` (the channel adapter's Web API + Socket Mode client).
- **Related**: `../OPERATOR_CREDENTIALS.md` (umbrella credentials guide), `../MESSENGER_CHANNELS.md` (multi-channel framework), `TELEGRAM.md` (the first live-channel guide), `../../design/PARTICIPANT_ATTRIBUTION.md` (§109 attribution / `HERALD_SLACK_OPERATOR_USERNAME`).

Sources verified 2026-05-31

Per HelixConstitution §11.4.99 + Herald §108.n (Latest-Source Documentation Cross-Reference Mandate). Every operator-facing instruction in this document was cross-referenced against the LATEST official online Slack documentation before publication. Slack moved its developer docs from `api.slack.com/*` reference pages to `docs.slack.dev/*` (302 redirects observed 2026-05-31); the `api.slack.com/apps` app-management console URL is unchanged.

Last verified: 2026-05-31

Source	URL	Used for
Slack app management console	https://api.slack.com/apps	§2 (Create New App → From scratch → name → workspace), §3 (OAuth & Permissions, Install/Reinstall to Workspace), §4 (Socket Mode toggle, App-Level Tokens).
Using Socket Mode (Events API)	https://docs.slack.dev/apis/events-api/using-socket-mode	§1 + §4 (Enable Socket Mode toggle wording; app level token prefix `xapp-`; generate under Basic

Source	URL	Used for
		Information → App-Level Tokens; events arrive ov the WebSocket; `apps.connections.open`).
Scope reference — `connections:write`	https://docs.slack.dev/reference/scopes/connections.write	§4 (the exact app-level token scope: "Grants permission to generate websocket URIs and conne Socket Mode").
Scope reference — `chat:write`	https://docs.slack.dev/reference/scopes/chat.write	§3 ("Send messages as your Slack app").
Scope reference — `chat:write.public`	https://docs.slack.dev/reference/scopes/chat.write.public	§3 + §5 ("Send messages to channels your Slack ap isn't a member of"; requires `chat:write`).
Scope reference — `channels:history`	https://docs.slack.dev/reference/scopes/channels.history	§3 (public-channel history; "View messages and ot content in public channels that your Slack app has been added to").
Scope reference — `groups:history`	https://docs.slack.dev/reference/scopes/groups.history	§3 (private-channel history equivalent).
Method reference — `chat.postMessage`	https://docs.slack.dev/reference/methods/chat.postMessage	§1 + §7 + §9 (required scope `chat:write`; `channel`/`text` args; error codes `channel_not_found`, `not_in_channel`, `is_archived`, `invalid_auth`, `not_authe` `missing_scope`).
Method reference — `auth.test`	https://docs.slack.dev/reference/methods/auth.test	§1 + §9 ("No scopes required"; returns `ok`/`url`/`te` `user`/`team_id`/`user_id`/`bot_id`; empty `user_id` ⇒ degenerate token).
Method reference — `conversations.history`	https://docs.slack.dev/reference/methods/conversations.history	§1 + §3 (requires one of `channels:history`/ `groups:history`/`im:history`/ `mpim:history`; `oldest` Unix-ts lower bound + `limit` default 100/max 999; errors `not_in_channel`/`channel_not_found`/ `missing_scope`).
Herald source (the authority on which env vars + methods are actually read)	`commons_messaging/channels/slack/*.go`, `qaherald/internal/messenger/slack.go`, `pherald/cmd/pherald/listen.go`, `scripts/e2e_bluff_hunt.sh`	Env-var names (`HERALD_SLACK_BOT_TOKEN`/ `HERALD_SLACK_APP_TOKEN`/ `HERALD_SLACK_CHANNEL_ID`), the E127 triple guard, token redaction (`sanitizeError` / `TestSlack_Send_ErrorDoesNotLeakTok` the `files.uploadV2`-vs-`files.upload` divergence.

Negative findings (documented honestly per §11.4.99(B)):

1. **No signing secret.** Herald uses **Socket Mode**, not HTTPS Events-API webhooks, so it reads **no** `HERALD_SLACK_SIGNING_SECRET`. The earlier placeholder reserved that name; it is not wired to anything. Do not expect it to have an effect.
2. **Default-channel var name.** The earlier placeholder used `HERALD_SLACK_DEFAULT_CHANNEL`; the **actual** code reads `HERALD_SLACK_CHANNEL_ID`. The placeholder name is dead.
3. `files.upload` **deprecation.** The qaherald QA client ships Slack's deprecated single-step `files.upload` (deprecated 2025); the production channel adapter already uses

`files.uploadV2`. E127 (text Send/reply) is unaffected; attachment upload via qaherald is the only impacted path.

4. **Scopes index page.** `docs.slack.dev/reference/scopes` is a table-of-contents only (no per-scope descriptions inline); the exact wording above was taken from each scope's own reference page (linked individually in this table), not from the index.